

Metriq

How the efficiency arena is built, one plane at a time. A step-by-step reading of the whole codebase, written so you can change it with confidence.



How to read this book

This is not the design document. The design document argued for what to build. This book explains what *was* built, why each piece is shaped the way it is, and where you put your hands when you want to change it.

Metriq is a working system with three rings of scope, all built and proven. Rather than list files, this book walks the system the way the data flows: from the one law that everything else obeys, out to the chain that enforces it, through the process that drives the race, to the screen a spectator watches. Every chapter ends where the next begins.

Four kinds of aside recur. Learn their colours and you can skim for exactly what you need:

THE WHY

The reasoning behind a decision that is not obvious from the code. When you are tempted to "simplify" something, read the nearest **why** first — it usually explains what breaks.

THE COUNCIL'S CALL

A real fork in the road, the options that were weighed, and the choice made. These are the load-bearing decisions; changing one ripples.

GOTCHA

A trap that already bit us, or one waiting for you. Several of these were only found by running the real system, not by reading the code.

TRY IT

A command you can run right now to see the thing the paragraph just described. The system is built to be pokeable without a chain or a key.

You do not need the chain, a wallet, or any paid service to follow along. Everything in this book can be run for free on your own machine. Where a step genuinely needs a testnet or an operator key, it is called out plainly.

CONTENTS

The map

01	Three planes, one race	The shape of the system and how a purchase travels through it.
02	The frozen core	Events, the shared reducer, and the metric with a trap in it.
03	The referee is a contract	Tallo and TaskRouter: money, caps, scoring, and the decline path.
04	Four minds, one budget	The seeded strategies and the knapsack that wins.
05	One process drives the field	The orchestrator, the write mutex, and the pacing lever.
06	The feed is the truth	The indexer, the bus, SSE, and why mock and chain are interchangeable.
07	The Pit Wall	Grid Fermé: the tower, the takeover, and four animated moments.
08	The narrator	The Copilot: pluggable, deterministic, never blank.
09	Every number resolves to a tx	Ring 2 Stubs: the receipt as a rendering rule.
10	Prove the win, hide the how	Ring 3: the real Noir circuit, proof, and on-chain verifier.
11	Running and tuning	Three modes, the tuning levers, and the demo.
12	Changing it	Where to add features, the deviations from the doc, and the decision log.

The one law

Metriq has exactly one non-negotiable rule, and every contract, event, and screen exists to honour it: **decline, never revert**. Understand this sentence and the rest of the system reads like a consequence of it.

The elimination that erased its own evidence

The product's most dramatic moment is a cap-out: an agent tries to buy one more subtask, has no budget left, and is eliminated live. The claim that makes it more than a cartoon is that this elimination is *on chain* — a real log a judge can open on the explorer. An earlier draft implemented it the obvious way, and the obvious way is broken:

THE PATH THAT ERASES ITS OWN EVIDENCE

```
// Tallo.charge()    a.active = false; emit CappedOut(...); return false;
// TaskRouter.pay() bool ok = tallo.charge(...); if (!ok) revert CappedOut();
```

On an EVM chain, **a reverted transaction persists no logs**. When `pay()` reverts, the whole transaction unwinds and every event emitted inside it — including the `CappedOut` the leaderboard is derived from — is discarded. The indexer never sees it. The only thing that "knows" an agent was eliminated is the off-chain runner's local catch. That is exactly the server-trusting state machine the design swore off: the drama would live in a variable, not on the chain.

GOTCHA

This is not a style preference. A revert-on-overspend path makes the sentence "eliminations are derived from on-chain logs" quietly *false*. Every later guarantee (the ledger reconciles from the chain, every number resolves to a tx) collapses if the elimination event does not persist.

The fix, and why it is everywhere

The correct path *declines*. On a cap breach, `charge()` deactivates the account, emits the event, and returns `false` — it does not revert. The router mirrors that with its own `PaymentDeclined` and also returns `false`. The state change and the logs both persist. The event is the elimination beat, on chain, for real.

THE WHY

Returning a boolean instead of reverting turns "failure" into "information." A declined charge is not an error the system recovers from; it is a first-class outcome the system reports. Once you internalise that, the whole event stream makes sense: it is a log of outcomes, and outcomes never throw.

You will see this shape in `Tallo.charge`, in `TaskRouter.pay`, in the `declined` and `capped` members of the event union, in the reducer's cap-out handler, and in the red takeover on the tower. They are all the same law, seen from different planes. When you add a new spend path, ask first: does it decline, or does it revert? Only one answer is allowed.

Three planes, one race

Metriq is a thin, verifiable core split across three planes. Money and scores live on the chain. An off-chain process runs the race and narrates it. The client is a live document. The trick that keeps it honest: the client renders *indexed chain events*, never the backend's own arithmetic.

THE THREE PLANES

```
CLIENT  - "Grid Ferme" (Next.js, React, SSE)
          the Pit Wall: timing tower + ledger rail, one screen
          ^ SSE: a typed, ordered, replayable RunEvent stream
OFF-CHAIN - one Node process, four logical services
          host (task + SSE + copilot) · indexer · orchestrator
          ^ viem reads + event logs          | OKX Payment SDK / pay()
ON-CHAIN  - X Layer (OKX zkEVM L2), Solidity
          Tallo.charge() DECLINES, never reverts
          TaskRouter.pay() emits Stub / PaymentDeclined
          RevealVerifier (Ring 3) verifies a Noir proof
```

Who is allowed to be trusted

CONCERN	SOURCE OF TRUTH	HOW YOU CHECK IT
Money, caps, eliminations	The chain (contract state + events)	Open the tx on the explorer
Quality of work	The operator's published rubric	Recompute it; it is deterministic
The leaderboard	A pure fold over chain events	Re-run the reducer on the same events

The frontend holds **no game logic**. It cannot decide who won; it can only render what the chain already decided. This is why the system "fails safe": restart the client and it re-derives every standing from the log; restart the off-chain process and it rebuilds from the chain; the task host can be slow or offline and the standings still reconcile.

The life of one purchase

Follow a single premium ANALYZE buy all the way through, because every feature is a variation on this path:

#	PLANE	WHAT HAPPENS
1	Orchestrator	The adaptive strategy decides <code>{taskId:"3", tier:"PREMIUM"}</code> .
2	Orchestrator	Serialises the write through the operator key and calls <code>router.pay(...)</code> .
3	Chain	<code>pay</code> calls <code>tallo.charge</code> ; under cap, funds move, <code>Charged + Stub</code> emit in one tx.
4	Indexer	<code>watchContractEvent</code> maps the two logs to a <code>charged</code> and a <code>stub</code> <code>RunEvent</code> onto the bus.
5	Host	The SSE route pushes both events to every connected client.
6	Client	The reducer folds them: spend ticks down, score ticks up, value/OKB recomputes, the tower re-sorts, a <code>Stub</code> receipt tears off.

THE WHY: FIELD SIZE IS A PARAMETER, NOT A CONSTANT

The contracts treat the field as N accounts against one task. Four is the demo default; two is a fast duel; six is the legible ceiling. The field is a single array (`FIELD_ORDER` in the core). Cutting to a two-agent duel under deadline is editing that array, not refactoring anything.

The frozen core

One small package, `@metriq/core`, is the spine. It defines the shape of a run event, the pure function that folds events into standings, and the metric everything is ranked on. The backend and the frontend both import it, so the number on the tower and the number the narrator speaks are *the same number by construction*. This package is frozen: change its shapes only with great care.

RunEvent: the one wire contract

Everything that ever happens in a run is one of these. The mock emits them; the real indexer emits them; they are identical either way. That interchangeability is the reason the whole UI can be built before a single contract is deployed.

PACKAGES/CORE/SRC/EVENTS.TS

```
export type RunEvent =
  | { kind:"opened";    ts; agentId; cap }
  | { kind:"stub";      ts; agentId; taskId; tier; price; score; cumulativeScore; txHash? }
  | { kind:"charged";   ts; agentId; amount; remaining }
  | { kind:"declined";  ts; agentId; taskId; attempted; remaining }
  | { kind:"capped";    ts; agentId; totalSpent }
  | { kind:"finished";  ts; agentId; totalScore }
  | { kind:"revealed";  ts; agentId; ok }
  | { kind:"feed-lost"; ts };
```

THE COUNCIL'S CALL: ONE ADDITIVE FIELD

The design's event union carried no transaction hash anywhere, yet the Ring 2 receipt's serial is a tx hash. Rather than build a second data path later, we added one optional field — `txHash?` on the `stub` event — while the shape was being frozen. Ring 1 ignores it; the mock supplies a display serial; Ring 2 uses it. One data path, decided up front.

The reducer: a pure fold, shared by two planes

`reduceRun(state, event)` is an ordinary reducer: give it the current standings and one event, get the next standings. It is pure — no clock reads, no randomness, no I/O — which is why it can run identically in a browser and on a server. Folding the whole event list gives the final standings; folding it one event at a time drives the live animation.

Each event nudges one lane. A `charged` lowers spend and remaining; a `stub` raises the cumulative score and tears off a receipt; a `capped` flips the lane to eliminated and *freezes* its metric so it never recomputes again. After every event the reducer re-ranks the lanes and folds a deterministic narrator line, so the state object is always complete.

GOTCHA FOUND ON A REAL CHAIN

The cap-out reason ("ELIMINATED at PUBLISH") reads the lane's last task. In the mock, events were pre-ordered so the declined task arrived first. On a real chain the indexer's two watchers (CappedOut from Tallo, PaymentDeclined from the router) can arrive in *either* order — so sometimes the reason read the stale task and said "at VERIFY". The fix: the declined handler also reconciles the reason if the lane is already eliminated, so the fold is order-independent. This bug was invisible until we ran the anvil pipeline.

The metric, and the trap inside it

Agents are ranked on **value per OKB**: quality points divided by spend. It lives in one file so it can never drift:

PACKAGES/CORE/SRC/METRIC.TS

```
export function valuePerOkb(qualityPoints, spentWei) {  
  if (spentWei === 0n) return 0;  
  return qualityPoints / (Number(spentWei) / 1e18);  
}
```

The trap: an agent that barely spends can post the *highest* raw value per OKB while producing the *lowest* total quality. The Conservative strategy does exactly this. If the tower ranked on raw efficiency, the agent that did the least would win — a bad story and a wrong one.

THE WHY: A TWO-KEY RANK, NOT ONE

The tower ranks in two keys. First: did the lane reach the quality target? Target-reachers sort above non-reachers. Only *then*, within each group, does value per OKB decide. Conservative posts the best raw efficiency but never clears the target, so it is pinned below Adaptive and Balanced who did. Eliminated lanes are pinned last with their metric frozen and struck through. The winner leads on delivered value, not on doing the least.

TRY IT

`pnpm --filter @metriq/core gen:round` folds the scripted round through this exact reducer and prints the standings. Watch Conservative land at P3 with the highest val/OKB — that is the trap being defused, live.

The referee is a contract

Two contracts hold the truth. **Tallo** owns money and the cap. **TaskRouter** owns the task table and the score ledger. Splitting them keeps the money contract small and auditable while the task logic evolves freely. Both obey the one law.

Tallo: the ledger that declines

Each agent has an `Account` with a cap, cumulative spend, and an active flag. Funding is dead simple: `openAccount` is `payable`, and the value sent is the cap — fund equals cap, no approval dance. The heart is `charge`:

CONTRACTS/SRC/TALLO.SOL — THE DECLINE PATH

```
function charge(bytes32 roundId, bytes32 agentId, uint128 amount)
    external onlyRouter returns (bool ok)
{
    Account storage a = _accounts[roundId][agentId];
    require(a.exists && a.active, "missing/inactive");
    uint128 remaining = a.cap - a.spent;
    if (amount > remaining) {
        a.active = false;                // eliminate
        emit CapDeclined(roundId, agentId, amount, remaining);
        emit CappedOut(roundId, agentId, a.spent);
        return false;                    // decline, do not revert
    }
    a.spent += amount;
    (bool sent,) = treasury.call{ value: amount }("");
    require(sent, "transfer failed");
    emit Charged(roundId, agentId, amount, a.spent, a.cap - a.spent);
    return true;
}
```

THE COUNCIL'S CALL: SET-ONCE ROUTER, NOT IMMUTABLE

Tallo needs the router's address and the router needs Tallo's — a deploy cycle. Making both `immutable` is a deadlock unless you predict addresses with `CREATE2`, which is fiddly and error-prone in a deploy script. Instead `Tallo.router` is set once via `initRouter()` behind a zero-address guard: the same freeze guarantee, a trivial two-step deploy. On the one-true-deadline charge path, reliable beats clever.

TaskRouter: scoring on chain, in one write

The one paid entrypoint is `pay`. It checks the caller is the agent's controller and the task is not already done, computes the tier price, calls `tallo.charge`, and — on success — accrues score from the posted table and emits a single `Stub` carrying price, score, and cumulative. One `pay` equals one on-chain

receipt. On a decline it mirrors with `PaymentDeclined` and returns false. There is no separate "report score" round-trip; scoring is on chain from the posted table.

THE COUNCIL'S CALL: A TABLE-DRIVEN BONUS

The rubric gives PUBLISH a +5 bonus *if* VERIFY was bought — a conditional the design's flat lookup could not express. Dropping the bonus made the target margins razor-thin and killed VERIFY's whole purpose. So the `Task` struct gained two fields, `bonusTaskId` and `bonusScore`: if that task was completed, award the bonus. Scoring stays fully on chain and table-driven, and VERIFY earns its place in the strategy.

TRY IT

`cd contracts && forge test runs 20 tests`. The one that matters most, `test_charge_capBreach_declines_withoutReverting`, proves the `CappedOut` log survives without a revert — the one law, asserted. A 512-run fuzz hammers the cap boundary.

Four minds, one budget

The competitors are one base agent running four tuned spend policies. A strategy is a *pure function of run context* — no LLM in the decision loop — which is what makes the demo reproducible take after take. The race is close by construction, and the cap-out lands on cue.

The four seeds

STRATEGY	POLICY	ROLE IN THE DEMO
Adaptive	bounded 0/1 knapsack over remaining budget	the champion, decided late
Balanced	best value/\$ first, one planned ENRICH splurge	keeps P1 honest, near miss
Conservative	cheapest required first, hoards budget	finishes safe, never leads on total
Greedy	premium everything, no guard	the elimination beat

THE WHY: GREEDY WALKS IN ORDER

The design's one-line Greedy sorted by raw score, which — worked through — actually caps out at VERIFY, not PUBLISH, breaking the scripted beat. Greedy's essence is "premium everything, no self-preservation," so it is implemented as buying the premium tier of every subtask *in order*. The required PUBLISH is therefore attempted last, which is exactly where the cap stops it. The demo beat is load-bearing; the exact sort expression is not.

The knapsack: the interview-impressive artifact

Adaptive plans rather than reacts. Each tick it runs a bounded 0/1 knapsack over the remaining budget, entering both tiers of a task as separate items and bounding the pool to `horizon + 3` so it recomputes cheaply. It is a real optimiser, not a lookup:

PACKAGES/STRATEGIES/SRC/KNAPSACK.TS (SHAPE)

```
items = tasks.flatMap(t => t.premiumPrice !== null
  ? [{task,premium:false},{task,premium:true}] : [{task,premium:false}]);
items.sort(by value-per-okb desc);
pool = items.slice(0, horizon + 3);           // bounded, cheap per tick
walk(i, spent, acc, score):                   // 0/1 branch: take / skip
  if score > bestScore: best = acc
  if not used[task] and spent+price <= budget: take
  skip
```

Its sequencing has one subtlety: when the plan includes VERIFY, Adaptive buys it *before* PUBLISH so the +5 bonus lands. That is why Adaptive's 88 beats Conservative's 57 even though Conservative barely

spends.

TRY IT — TUNING DAY IN ONE LINE

`pnpm --filter @metriq/strategies simulate` runs all four through the real decline mechanics and prints the invariants. Override prices to explore: `ANALYZE_PREMIUM=0.32 pnpm --filter @metriq/strategies simulate` and watch the Adaptive-vs-Conservative gap blow from 0.40 to 5.56 — the tool rejects the over-tune.

One process drives the field

The orchestrator is the only writer. It holds the operator key (which owns the controller role, so it signs every `pay`), opens and stakes identical budgets, and then advances every lane independently — while serialising the actual contract writes.

Serialise the writes, parallelise the waiting

Every lane runs its own loop inside one `Promise.all`: `decide`, `pay`, `wait`, `repeat`. But all those lanes share one operator key, and one key means one nonce sequence. Fire two transactions concurrently and they collide on the same nonce.

THE WHY: A TINY WRITE MUTEX

The rule from the design — "serialise contract writes through the operator key and parallelise the waiting" — is enforced by a one-object mutex. Every `pay` / `open` / `finish` acquires the lock just long enough to *send* the transaction (claim the next nonce); waiting for the receipt happens outside the lock, in parallel. So the field advances concurrently but the nonces stay strictly ordered.

APPS/ORCHESTRATOR/SRC/UTIL.TS (SHAPE)

```
class Mutex {
  private tail = Promise.resolve();
  run(fn) { const r = this.tail.then(fn, fn);
    this.tail = r.then(()=>{}, ()=>{}); return r; }
}
```

The pacing lever

Between decisions each lane sleeps a jittered 300–900 ms. That stagger is what makes the tower visibly re-sort instead of snapping. The jitter is a *seeded* PRNG (`mulberry32`), so the pacing reproduces take after take — pin `JITTER_SEED` and the same run plays back identically. Decisions come from the pure strategy; only the timing is random, and even that is deterministic.

GOTCHA

The lane loop reads the account's `active` flag each tick and breaks if false. A declined `pay` also returns false and breaks the loop. Either way the lane stops cleanly — but note `finish` is still called for eliminated lanes, and the reducer *ignores* a `finished` event on an already-eliminated lane. Do not "fix" that by skipping `finish`; the reducer's guard is intentional.

The feed is the truth

Everything off chain is stateless over the log. The indexer turns chain logs into `RunEvents`, a bus keeps an ordered replay per round, and SSE streams them to the client. The mock and the real feed emit the identical stream, so swapping them is a transport change with zero UI change.

The one indirection that makes mock-first work

The client subscribes through a two-line interface, `RunSource`. There are two implementations: `MockRunSource` replays a canned JSON round on a timer; `SseRunSource` wraps `EventSource`. The composition root — one file, one line — chooses which. Everything downstream reads the same reducer, so the choice is invisible below it.

APPS/WEB/SRC/APP/PAGE.TSX (THE SWAP POINT)

```
const source = FEED === "sse"
  ? createSseRunSource(ROUND_ID)
  : createMockRunSource({ speed: 1 });
```

The indexer: backfill, then watch, deduped

The indexer maps six events (opened, charged, stub, declined→declined, capped, finished) plus the optional Ring 3 reveal. A naive watcher only sees events emitted after it starts — so a client that connects after the round already ran would get an empty replay.

THE WHY: BACKFILL + DEDUP

On start, the indexer first pulls the round's whole history with `getContractEvents`, sorts it into chain order, and pushes it; then it goes live with `watchContractEvent`. A `seen` set keyed on `(blockNumber, logIndex)` makes the boundary between backfill and live never double-count. This is why a late SSE joiner still gets the entire story.

The bus stamps each event's timestamp relative to the round's first event, matching the mock's `atMs` semantics exactly — so the client's clock reads the same whether the source is canned or chain. That symmetry is not an accident; it is the whole point.

TRY IT — THE WHOLE PIPELINE, NO TESTNET

`bash scripts/e2e-sse.sh` starts anvil, deploys the contracts, runs the host (indexer + SSE) and the orchestrator, connects an SSE client *before* the run, and folds the streamed events with the same reducer the frontend uses. Fifty-four events over HTTP, same standings, greedy out at PUBLISH.

The Pit Wall

The frontend is one screen: a timing tower on the left (the sport), a ledger rail on the right (the statement), one ink rule between them. It is a live sporting document, not a dashboard. The signature is F1 timing colour on graph paper.

Grid Fermé, in tokens

Purple means best, green means improving, red means out, mapped onto value per dollar — a vocabulary a spectator reads in half a second. Everything else is ink on paper. One bold idea; everything around it quiet. The lane accent is a pure function of state:

APPS/WEB/SRC/LIB/ACCENT.TS

```
eliminated ? red           // out
: rank === 1 ? purple      // best
: delta > 0 ? green        // improving
: ink;                     // neutral
```

Exactly four animated moments

Budget drain, tower re-sort, cap-out takeover, final lock. No fifth. The re-sort is a 220 ms physical translate via `motion` layout, not a celebration. The one place boldness is spent is the cap-out: the lane turns full red, reads **ELIMINATED at PUBLISH**, the metric freezes struck through, and the survivors re-sort. `prefers-reduced-motion` collapses all four to crossfades.

GOTCHA: PAPER ON RED, NOT RED ON RED

The eliminated lane is a full red bar. The first pass coloured its text dark red too — invisible. The contrast audit specifies *paper* text on lane-red (5.1:1). The lesson generalises: every colour state also carries a text-label twin, so "eliminated" is written, not just shown. That is the accessibility floor, and it is not optional.

The tower is an ARIA table with `aria-live="polite"`, all money in tabular figures, focus rings on keyboard nav. Tablet collapses the rail to a bottom sheet; mobile compresses lanes to position, name, and value per OKB. The tower is sacred and never disappears at any width.

The narrator

The Copilot explains, in one sentence, why the order is what it is. It is a narrator, never a judge: quality comes from the rubric, money from the chain, and the model only translates state into a line a human reads in two seconds. Its defining property is that it *never goes blank*.

Deterministic first, live in front

The reducer folds a deterministic line from the same state the tower shows — "Adaptive leads on value per OKB, Greedy capped chasing raw score." That line needs no key, no network, and cannot disagree with the tower because it reads the same lanes. The live model call, when configured, sits *in front* of it: try the model, and on any error, timeout, or empty parse, fall back to the deterministic line.

THE COUNCIL'S CALL: A PLUGGABLE NARRATOR

The operator had no budget for a paid API, so the Copilot became provider-pluggable. It picks, in order: an OpenAI-compatible endpoint (`COPILOT_BASE_URL` — free Groq or Google AI Studio tiers, or a local Ollama), then an Anthropic key, else the deterministic line. The default needs nothing and runs offline. This is a justified deviation: the Copilot is a provider-agnostic narrator; the product's real agents are the four strategies, not this line.

On the client, a `useCopilot` hook debounces to at most one call per four seconds so inference never stalls the demo, and it only calls out when explicitly switched to live mode. Otherwise it simply shows the reducer's line. In the recording, the deterministic path is the safe choice — reproducible, instant, and honest.

Every number resolves to a tx

A Stub is not a new contract. It is a *rendering rule* over events that already exist. This is what upgrades "trust the leaderboard" to "verify the leaderboard": every figure on screen resolves to a transaction a judge can open on the explorer.

One completed subtask emits a `stub` event (with its paired `charged`). The pure reducer `foldStubs` turns each into a receipt whose serial is the tx hash, with the cost, the quality earned, and the remaining balance. In the ledger rail each receipt is a torn-edge ticket — the perforation is the product's signature carried into the receipt — and the serial links out to the X Layer explorer.

THE WHY: NO NEW TRUST SURFACE

Because the Stub is derived from the same events the tower folds, it cannot say anything the tower does not. It adds verifiability without adding a place to be wrong. That is why Ring 2 could ship as a rendering change, not a contract change.

This is also where the earlier `txHash` decision pays off: the serial is right there in the event, one data path, no reconciliation. Cutting Ring 2 would remove the receipts and leave the tower untouched — the ledger rail simply shows less.

Prove the win, hide the how

Ring 3 is the optional flourish, and it is real. At settle, the winner posts one Noir proof: it reached the quality target under the cap via a route matching a commitment posted before the run — *without revealing the route*. This is not a mock. The proof is generated by Barretenberg and verified on chain.

The circuit

The private witness is the route (which tier of each subtask), its prices, and its scores, plus a salt. The public inputs are what the chain already knows: the commitment, the cap, the spend, the quality target. Three assertions carry the claim:

CIRCUITS/REVEAL/SRC/MAIN.NR (THE THREE CHECKS)

```
let h = std::hash::pedersen_hash(preimage);
assert(h == commitment);           // 1) plan matches the commitment
assert(total == spent);             // 2) prices sum to public spend...
assert(total as u64 <= cap as u64); // ...within cap
assert(earned as u64 >= quality_target); // 3) the hidden route met the target
```

THE COUNCIL'S CALL: HARDEN THE COMMITMENT

The design's circuit committed only route + salt, which would let a prover keep the committed route but swap in inflated scores. Our commitment binds route *and prices and scores*. A small change; a sound proof instead of a plausible one.

From circuit to on-chain green tick

The pipeline is real and reproducible: `nargo compile` and `execute` produce the witness; `bb prove` and `write_vk` produce an 8 KB UltraHonk proof and key; `bb write_solidity_verifier` emits a ~100 KB Solidity verifier whose `verify(bytes, bytes32[])` exactly matches the `INoirVerifier` interface `RevealVerifier` already expected. The proof verifies off chain (`bb verify`) and, more importantly, *on chain*: a Foundry test deploys the verifier, reads the committed proof fixture, and confirms it — while a tampered public input is rejected.

GOTCHA: THE VERIFIER NEEDS A NEWER COMPILER

The generated verifier uses `require(bool, CustomError())`, a Solidity 0.8.26+ feature, so Foundry pins 0.8.27 even though our own contracts stay ^0.8.24. The verifier also exceeds the EIP-170 code-size limit, so a mainnet deploy needs it split; on anvil and X Layer testnet it deploys as-is.

On screen this is one component: `ProofTick`, the green **PROVEN** badge that lands on the winner lane when the reveal is indexed. Cutting Ring 3 deletes that one file and drops one address from the indexer config. Nothing else changes — that is the cut-line discipline the design promised, honoured exactly.

TRY IT

`cd contracts && forge test --match-path test/Reveal.t.sol` runs the three Ring 3 tests: the real proof verifies on chain, the reveal emits the green tick, and a lied-about spend is rejected. The proof is committed, so this runs on a fresh clone with no Noir toolchain.

Running and tuning

Three ways to run it, from zero-setup to fully on chain, all free. The full instructions live in `DEMO.md`; the levers in `TUNING.md`. This chapter is the orientation.

MODE	COMMAND	NEEDS
Mock (the safe run)	<code>pnpm --filter @metriq/web dev</code>	nothing
Full local pipeline	<code>bash scripts/e2e-sse.sh</code>	Foundry (anvil)
X Layer testnet	<code>forge script Deploy ... --broadcast</code>	a funded key, OKX SDK

Tuning, made data-driven

The race is close by construction, not luck. Four invariants must hold: Greedy caps at PUBLISH, Adaptive finishes first among target-reachers, Conservative misses the target, and the Adaptive-vs-Conservative value gap stays within about two. The simulator prints all four and fails loudly if a change breaks them. The levers are the premium prices on ENRICH and ANALYZE, the quality target, the knapsack horizon and reserve, and the jitter seed.

THE WHY: PRICES LIVE IN TWO LANGUAGES

The task table is duplicated between the TypeScript core and the Solidity deploy script, because Solidity cannot import a TS table. When you tune a price, change it in `packages/core/src/tasks.ts` and `contracts/script/Deploy.s.sol`, then re-run `gen:round` and `simulate` to confirm the drama still lands. The duplication is a known cost of the two-plane design; the simulator is the guard that catches drift.

The 90-second script — beat by beat, with what to say — is in `DEMO.md`. The discipline it enforces: if anything wobbles live, cut to the mock. The contracts and events stay real; only the pacing is pinned.

Changing it

You now know the system as data flowing through three planes. This last chapter is a map of where to put your hands, the decisions that are load-bearing, and the deviations from the original design and why they were made.

Where features go

YOU WANT TO...	START HERE
Add a subtask or change prices	<code>packages/core/src/tasks.ts</code> + <code>contracts/script/Deploy.s.sol</code> , then <code>simulate</code>
Add a fifth strategy	<code>packages/strategies/src/policies.ts</code> ; register it in <code>FIELD</code>
Change the field size	<code>FIELD_ORDER</code> in <code>packages/core/src/agents.ts</code>
Add a new on-chain event to the UI	extend the frozen <code>RunEvent</code> union, teach the reducer, add an indexer spec
Restyle without breaking state	the CSS tokens in <code>apps/web/src/styles/tokens.css</code>
Swap the narrator model	env only: <code>COPILOT_BASE_URL</code> / <code>COPILOT_MODEL</code>

GOTCHA: THE FROZEN CORE IS FROZEN FOR A REASON

The `RunEvent` union and the reducer's `RunState` shape are the contract between every plane. Change a field and you touch the mock JSON, the indexer mappers, the reducer, and the components at once. That is fine when intended — just do all of it in one change, and re-run `gen:round` to confirm the mock still folds to the scripted drama.

The deviations from the design, and why

CHANGE	WHY
txHash? on the stub event	Ring 2's serial is a tx hash; one data path, decided while freezing the shape
Set-once router, not immutable	breaks the deploy cycle without a CREATE2 dance
Table-driven VERIFY bonus	keeps conditional scoring on chain; preserves VERIFY's purpose
Greedy walks in order	lands the cap-out at PUBLISH, the scripted beat
Vanilla CSS, not Tailwind	bespoke UI; zero fragile dependency; tokens are already CSS
Pluggable / free Copilot	no-budget operator; deterministic default is offline-safe
Hardened reveal commitment	binds prices and scores, not just the route — a sound proof
solc 0.8.27 for the build	the generated verifier needs 0.8.26+; our contracts stay ^0.8.24

The repository, at a glance

```

contracts/      Tallo, TaskRouter, RevealVerifier, HonkVerifier (Foundry, 20 tests)
circuits/reveal/ the Noir circuit + Prover.toml (Ring 3)
packages/core/   FROZEN: events, reducer, metric, stub fold, domain tables
packages/strategies/ the four seeds + the knapsack
packages/chain/  shared viem ABIs, id encoding, clients, indexer, bus
apps/web/        the Pit Wall (Grid Ferme)
apps/host/       task host, SSE feed, copilot, indexer runtime
apps/orchestrator/ drives the field, holds the operator key
scripts/         e2e.sh, e2e-sse.sh (anvil proofs)
DEMO.md TUNING.md README.md  the runbooks

```

THE LAST WHY

Every decision in this book bent toward one property: the number on the screen is the number on the chain, and you can prove it. Keep that property and you can change almost anything. Break it — trust the backend's math, revert on overspend, let the tower invent a standing — and you have a nice animation of a thing that is not true. The whole system is a machine for staying honest under load. Build on it in that spirit.

The result stands

This manual describes Metriq as built: three rings, all proven, all runnable for free. Ring 1 is the whole game — accounts, cap, router, live tower, clean cap-out, narrator — proven end to end on a real EVM. Ring 2 makes every number resolve to a transaction. Ring 3 lets the winner prove its efficiency in zero knowledge, verified on chain.

What remains is not construction. It is the operator's: the X Layer testnet deploy with the OKX Payment SDK, the ninety-second recording, and the submission. The machine is ready.
